

more protection against accidental access to shared data. However, if processes need to communicate a lot, we may see a noticeable adverse affect on performance in comparison to using threads.

3.2 Virtualization

Threads and processes can be seen as a way to do more things at the same time. In effect, they allow us to build (pieces of) programs that appear to be executed simultaneously. On a single-processor computer, this simultaneous execution is, of course, an illusion. As there is only a single CPU, only an instruction from a single thread or process will be executed at a time. By rapidly switching between threads and processes, the illusion of parallelism is created.

This separation between having a single CPU and being able to pretend there are more can be extended to other resources as well, leading to what is known as **resource virtualization**. This virtualization has been applied for many decades, but has received renewed interest as (distributed) computer systems have become more commonplace and complex, leading to the situation that application software is mostly always outliving its underlying systems software and hardware.

Principle of virtualization

In practice, every (distributed) computer system offers a programming interface to higher-level software, as shown in Figure 3.6(a). There are many different types of interfaces, ranging from the basic instruction set as offered by a CPU to the vast collection of application programming interfaces that are shipped with many current middleware systems. In its essence, **virtualization** deals with extending or replacing an existing interface so as to mimic the behavior of another system, as shown in Figure 3.6(b). We will come to discuss technical details on virtualization shortly, but let us first concentrate on why virtualization is important.

Virtualization and distributed systems

One of the most important reasons for introducing virtualization back in the 1970s, was to allow legacy software to run on expensive mainframe hardware. The software not only included various applications, but in fact also the operating systems they were developed for. This approach toward supporting legacy software has been successfully applied on the IBM 370 mainframes (and their successors) that offered a virtual machine to which different operating systems had been ported.

As hardware became cheaper, computers became more powerful, and the number of different operating system flavors was reducing, virtualization

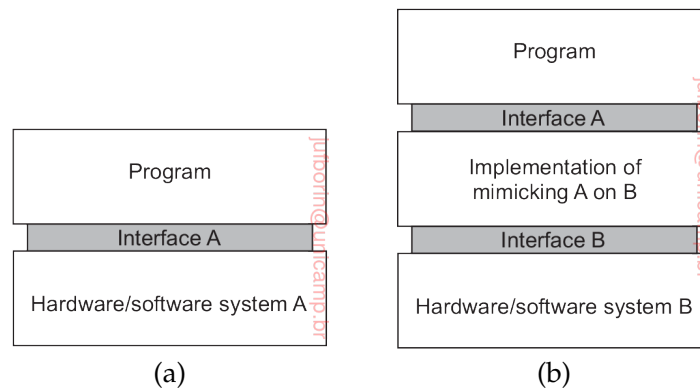


Figure 3.6: (a) General organization between a program, interface, and system. (b) General organization of virtualizing system A on top of B.

became less of an issue. However, matters have changed again since the late 1990s. First, while hardware and low-level systems software change reasonably fast, software at higher levels of abstraction (e.g., middleware and applications), are often much more stable. In other words, we are facing the situation that legacy software cannot be maintained in the same pace as the platforms it relies on. Virtualization can help here by porting the legacy interfaces to the new platforms and thus immediately opening up the latter for large classes of existing programs.

Note 3.4 (Discussion: Stable software?)

Although there is indeed a lot of legacy software that can benefit from stable interfaces to rapidly changing underlying hardware, it is a mistake to believe that the software for widely available services hardly changes. With the increasing shift toward server-side computing in the form of **Software-as-a-Service (SaaS)**, much software can be maintained for a relatively homogeneous platform, owned entirely by the organization offering the associated service. As a consequence, maintaining software products can be much easier, as there is much lesser need to distribute changes to potentially millions of customers. In fact, changes may rapidly succeed each other following changes in available hardware and platform, but without any client actually noticing downtimes [Barroso and Hölze, 2009].

Equally important is the fact that networking has become completely pervasive. It is hard to imagine that a modern computer is not connected to a network. In practice, this connectivity requires that system administrators maintain a large and heterogeneous collection of server computers, each one running very different applications, which can be accessed by clients. At the same time the various resources should be easily accessible to these applications. Virtualization can help a lot: the diversity of platforms and

machines can be reduced by essentially letting each application run on its own virtual machine, possibly including the related libraries *and* operating system, which, in turn, run on a common platform.

This last type of virtualization provides a high degree of portability and flexibility. For example, in order to realize content delivery networks that can easily support replication of dynamic content, Awadallah and Rosenblum [2002] have argued that management becomes much easier if edge servers would support virtualization, allowing a complete site, including its environment to be dynamically copied. These arguments are still valid, and indeed, portability is perhaps the most important reason why virtualization plays such a key role in many distributed systems.

Types of virtualization

There are many different ways in which virtualization can be realized. An overview of these various approaches is described by Smith and Nair [2005a]. To understand the differences in virtualization, it is important to realize that computer systems generally offer four different types of interfaces, at three different levels:

1. An interface between the hardware and software, referred to as the **instruction set architecture (ISA)**, forming the set of machine instructions. This set is divided into two subsets:
 - Privileged instructions, which are allowed to be executed only by the operating system.
 - General instructions, which can be executed by any program.
2. An interface consisting of **system calls** as offered by an operating system.
3. An interface consisting of library calls, generally forming what is known as an **application programming interface (API)**. In many cases, the aforementioned system calls are hidden by an API.

These different types are shown in Figure 3.7. The essence of virtualization is to mimic the behavior of these interfaces.

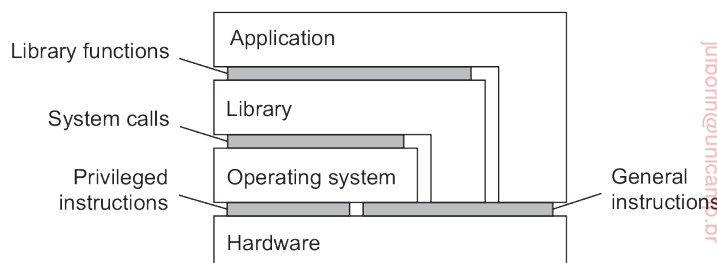


Figure 3.7: Various interfaces offered by computer systems.

Virtualization can take place in two different ways. First, we can build a runtime system that essentially provides an abstract instruction set that is to be used for executing applications. Instructions can be interpreted (as is the case for the Java runtime environment), but could also be emulated as is done for running Windows applications on Unix platforms. Note that in the latter case, the emulator will also have to mimic the behavior of system calls, which has proven to be generally far from trivial. This type of virtualization, shown in Figure 3.8(a), leads to what Smith and Nair [2005a] call a **process virtual machine**, stressing that virtualization is only for a single process.

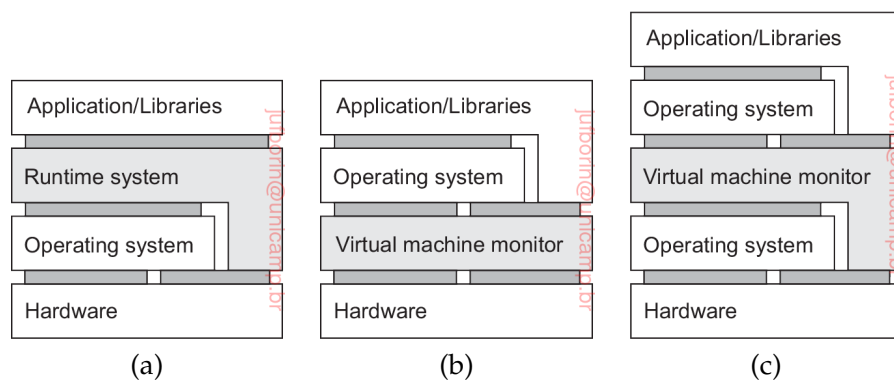


Figure 3.8: (a) A process virtual machine. (b) A native virtual machine monitor. (c) A hosted virtual machine monitor.

An alternative approach toward virtualization, shown in Figure 3.8(b), is to provide a system that is implemented as a layer shielding the original hardware, but offering the complete instruction set of that same (or other hardware) as an interface. This leads to what is known as a **native virtual machine monitor**. It is called native because it is implemented directly on top of the underlying hardware. Note that the interface offered by a virtual machine monitor can be offered *simultaneously* to different programs. As a result, it is now possible to have multiple, and different **guest operating systems** run independently and concurrently on the same platform.

A native virtual machine monitor will have to provide and regulate access to various resources, like external storage and networks. Like any operating system, this implies that it will have to implement device drivers for those resources. Rather than doing all this effort anew, a **hosted virtual machine monitor** will run on top of a trusted **host operating system** as shown in Figure 3.8(c). In this case, the virtual machine monitor can make use of existing facilities provided by that host operating system. It will generally have to be given special privileges instead of running as a user-level application. Using a hosted virtual machine monitor is highly popular in modern distributed systems such as data centers and clouds.

As argued by Rosenblum and Garfinkel [2005], virtual machines are becoming increasingly important in the context of reliability and security for (distributed) systems. As they allow for the isolation of a complete application and its environment, a failure caused by an error or security attack need no longer affect a complete machine. In addition, as we also mentioned before, portability is greatly improved as virtual machines provide a further decoupling between hardware and software, allowing a complete environment to be moved from one machine to another. We return to migration in Section 3.5.

Note 3.5 (Advanced: On the performance of virtual machines)

Virtual machines perform surprisingly well. In fact, many studies show that modern virtual machines perform close to running applications directly on the host operating system. Let us take a closer look at what is going under the hood of virtual machines. A detailed and comprehensive account of virtual machines is provided by Smith and Nair [2005b].

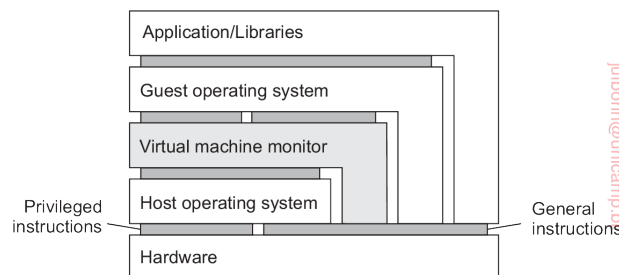


Figure 3.9: Applications, guest operating system, virtual machine monitor, and host operating system on a single hardware platform.

Part of the answer to performance issues is shown in Figure 3.9, which forms an extension of Figure 3.8(c): a large part of the code constituting a virtual machine monitor, guest operating system, and application is running natively on the underlying hardware. In particular, all general (i.e., unprivileged) machine instructions are directly executed by the underlying machine instead of being interpreted or emulated.

This approach is not new and is founded on research by Popek and Goldberg [1974] who formalized the requirements for the efficient execution of virtual machines. In a nutshell, Popek and Goldberg assumed that the underlying machine provided at least two modes of operation (system and user mode), that a subset of the instructions could be executed only in system mode, and that memory addressing was relative (i.e., a physical address was obtained by adding a relative address to an offset found in a relocation register). A distinction was further made between two types of instructions. A **privileged instruction** is an instruction that is characterized by the fact that if and only if executed in user mode, it causes a **trap** to the operating system. **Nonprivileged instructions** are all other instructions.

Given these formal assumptions, Popek and Goldberg defined two classes

of special instructions. A **control-sensitive instruction** is one that may affect the configuration of a machine. A typical example is an instruction that affects the memory layout, for example, by changing the memory offset as stored in a relocation register. Another example are instructions that affect the interrupt table containing pointers to interrupt handlers.

A **behavior-sensitive instruction** is one whose effect is partially determined by the context in which it is executed. For example, Intel x86 processors have instructions that may, or may not affect certain registers depending on whether that instruction is executed in system mode or user mode. An example given in [Smith and Nair, 2005b] is that of the POPF instruction, which may set an interrupt-enabled flag, but only when executed in system mode.

We now have the following important result:

For any conventional computer, a virtual machine monitor may be constructed if the set of sensitive instructions for that computer is a subset of the set of privileged instructions.

What this says is that as long as sensitive instructions are caught when executed in user mode, we can safely run all nonsensitive instructions natively on the underlying hardware. This also means that when designing instruction sets, if we take care that the above requirement is met, we will not be unnecessarily obstructing efficient virtualization of that instruction set.

Unfortunately, not all instruction sets have privileged-only sensitive instructions, including perhaps the most popular one, namely the Intel x86 instruction set. As it turns out, this set has 17 sensitive instructions that are not privileged [Robin and Irvine, 2000]. In other words, each of these instructions can be executed in user mode without causing a trap to the operating system, yet affect the way that the operating system is managing its resources. In these cases, there are essentially two solutions.

The first solution is to emulate all instructions. Of course, this would have a serious adverse effect on performance. To circumvent problems, an approach implemented in VMWare [Sugerman et al., 2001], is to scan the executable and to insert code around the nonprivileged sensitive instructions to divert control to the virtual machine monitor. There, appropriate emulation will take place, for example, by considering the context in which the instruction was to be executed. The effect is that full virtualization can take place, meaning that execution can take place without changing the guest operating system, nor the application itself.

An alternative solution is to apply **paravirtualization**, which requires the guest operating system to be modified. In particular, the guest operating system is modified such that all side effects of running nonprivileged sensitive instructions in user mode, which would normally be executed in system mode, are dealt with. For example, code can be rewritten such that these instructions simply no longer occur, or if they do, that their semantics are the same regardless whether being executed in user or system mode. Paravirtualization has been adopted by Xen [Barham et al., 2003; Chisnall, 2007].

Application of virtual machines to distributed systems

From the perspective of distributed systems, the most important application of virtualization lies in cloud computing. As we already mentioned in Section 1.3, cloud providers offer roughly three different types of services:

- **Infrastructure-as-a-Service (IaaS)** covering the basic infrastructure
- **Platform-as-a-Service (PaaS)** covering system-level services
- **Software-as-a-Service (SaaS)** containing actual applications

Virtualization plays a key role in IaaS. Instead of renting out a physical machine, a cloud provider will rent out a virtual machine (monitor) that may, or may not, be sharing a physical machine with other customers. The beauty of virtualization is that it allows for almost complete isolation between customers, who will indeed have the illusion that they have just rented a dedicated physical machine. Isolation is, however, never complete, if only for the fact that the actual physical resources are shared, in turn leading to observable lower performance.

To make matters concrete, let us consider the Amazon **Elastic Compute Cloud**, or simply **EC2**. EC2 allows one to create an environment consisting of several networked virtual servers, thus jointly forming the basis of a distributed system. To make life easy, there is a (large) number of pre-configured *machine images* available, referred to as **Amazon Machine Images**, or simply **AMIs**. An AMI is an installable software package consisting of an operating-system kernel along with a number of services. An example of a simple, basic AMI is a **LAMP** image, consisting of a Linux kernel, the Apache Web server, a MySQL database system, and PHP libraries. More elaborate images containing additional software are also available, as well as images based on other Unix kernels or Windows. In this sense, an AMI is essentially the same as a boot disk (although there are few important differences to which we return shortly).

An EC2 customer needs to select an AMI, possibly after adapting or configuring one. An AMI can then be *launched* resulting in what is called an **EC2 instance**: the actual virtual machine that can be used to host a customer's applications. An important issue is that a customer will hardly ever know exactly where an instance is actually being executed. Obviously, it is running on a single physical machine, but where that machine is located remains hidden. The closest one can get to pinpointing the location where an instance should run is by selecting one of a few regions provided by Amazon (US, South America, Europe, Asia).

To communicate, each instance obtains two IP addresses: a private one that can be used for internal communication between different instances, making use of EC2's internal networking facilities, and a public IP address allowing any Internet clients to contact an instance. The public address is mapped to the private one using standard **network-address translation (NAT)** technology.

A simple way to manage an instance is to make use of an SSH connection, for which Amazon provides the means for generating the appropriate keys.

The EC2 environment in which an instance is executed provides different levels of the following services:

- **CPU:** allows to select the number and type of cores, including GPUs
- **Memory:** defines how much main memory is allocated to an instance
- **Storage:** defines how much local storage is allocated
- **Platform:** distinguishes between 32-bit or 64-bit architectures
- **Networking:** sets the bandwidth capacity that can be used

In addition, extra resources can be requested such as an additional networking interface. The local storage that comes with an instance is *transient*: when the instance stops, all the data stored locally is lost. In order to prevent data loss, a customer will need to explicitly save data to persistent store, for example, by making use of Amazon's Simple Storage Service (S3). An alternative is to attach a storage device that is mapped to Amazon's **Elastic Block Store (Amazon EBS)**. Again, this is yet another service, but one that can be used in the form of a virtual block device that is simply mounted as one would mount an additional hard disk. When an instance is stopped, all data that was stored on EBS will persist. And just as one would expect, an EBS device can be (re)mounted to any other instance as well.

It should be clear by now that, without having gone into any significant level of detail, the IaaS as offered by EC2 allows a customer to create a (potentially large) number of virtual machines, each configured with resources as needed, and capable of exchanging messages through an IP network. In addition, these virtual machines can be accessed from anywhere over the Internet (provided a client has the proper credentials). As such, Amazon EC2, like many other IaaS providers, offers the means to configure a complete distributed system consisting of networked virtual servers and running customer-supplied distributed applications. At the same time, those customers will not need to maintain any physical machine, which by itself is often already a huge gain as we will encounter at several occasions throughout this text. One can indeed argue that virtualization lies at the core of modern cloud computing.

3.3 Clients

In the previous chapters we discussed the client-server model, the roles of clients and servers, and the ways they interact. Let us now take a closer look at the anatomy of clients and servers, respectively. We start in this section with a discussion of clients. Servers are discussed in the next section.